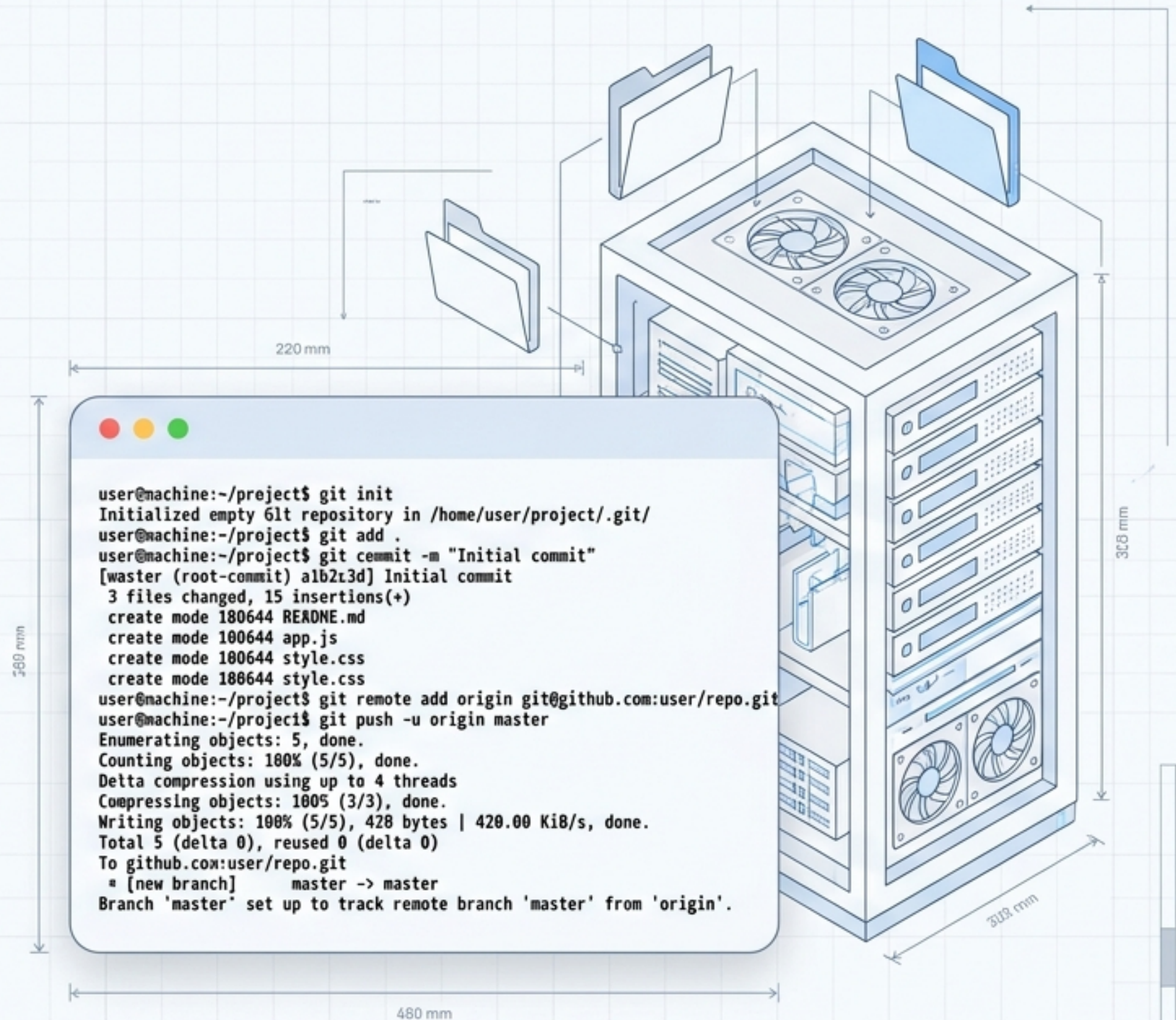


Dominando Git y GitHub desde la Terminal

Una guía visual y práctica.

- Aprende a configurar tu entorno local.
- Gestiona autenticación y claves SSH.
- Domina los flujos de Local a Remoto y de Remoto a Local.



El flujo de trabajo en Git atraviesa cuatro entornos distintos



El primer paso: Instalar el motor y establecer tu identidad

Fase 1: Instalación (Linux)

```
$ sudo apt install git  
$ git --version
```

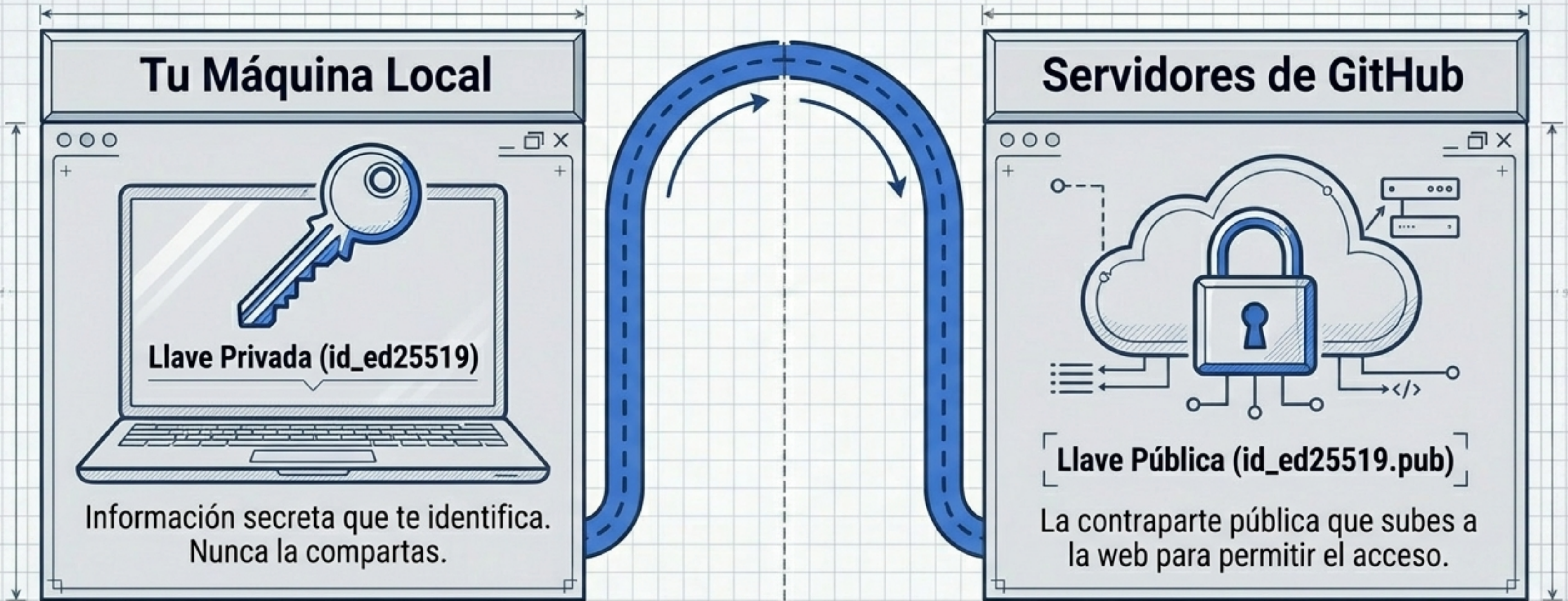
Comprueba que se ha instalado correctamente.

Fase 2: Configuración Global

```
$ git config --global  
user.name "Tu Nombre"  
$ git config --global  
user.email "tu@correo.com"
```

Nota: Esta identidad se grabará permanentemente en cada commit (cambio) que realices en cualquier proyecto de tu sistema.

La autenticación moderna exige un apretón de manos criptográfico



Importante: La validación por contraseña está obsoleta.
Este enlace seguro vía SSH es ahora obligatorio.

Genera el par de claves y actívalas en tu entorno local

Paso 1: Generar claves en tu máquina.

```
$ ssh-keygen -t ed25519 -C "tu@correo.com"
```

Crea los archivos en /home/usuario/.ssh

Paso 2: Activar el Agente SSH.

```
$ eval "$(ssh-agent -s)"
```

Despierta el sistema que maneja las claves en segundo plano.

Paso 3: Añadir la clave privada.

```
$ ssh-add ~/.ssh/id_ed25519
```

Conecta tu llave secreta al agente.



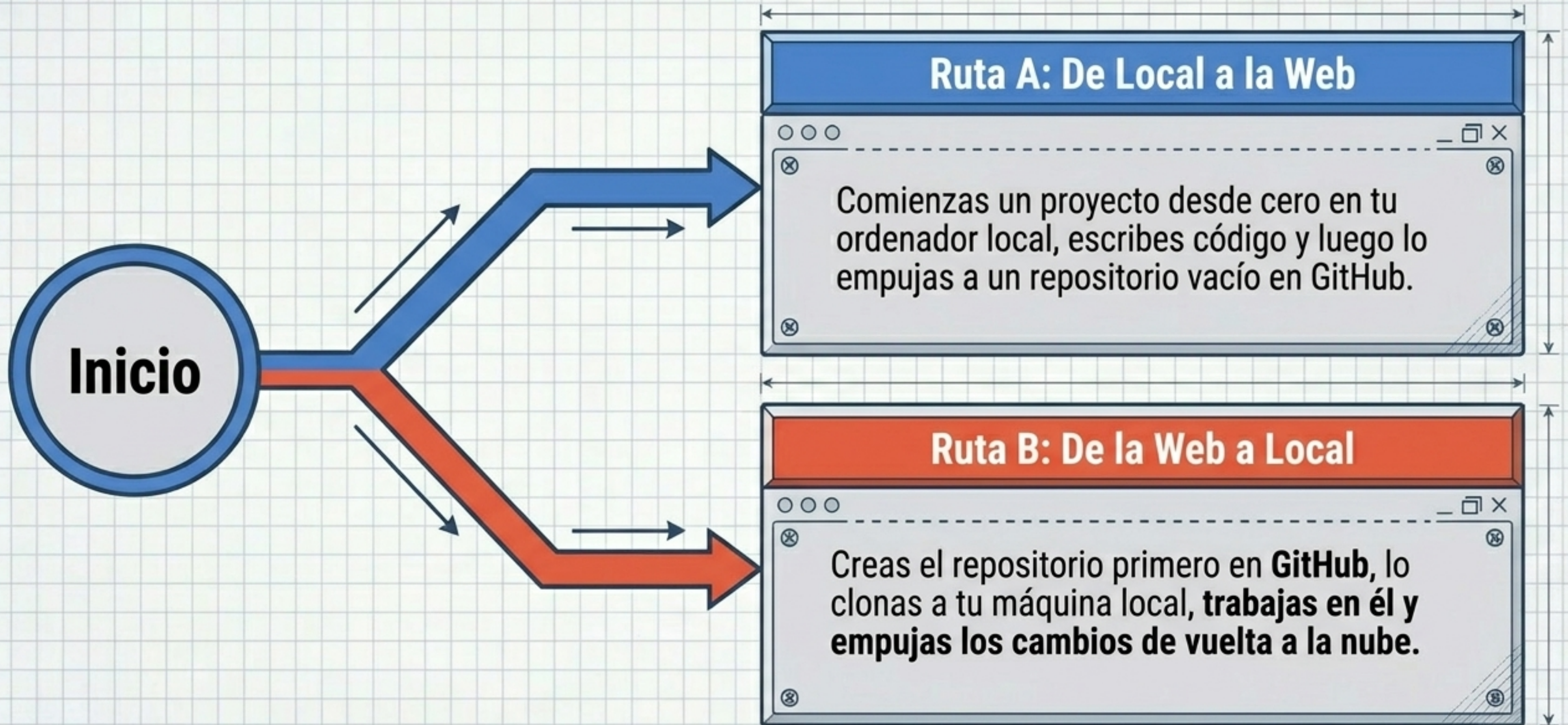
Para vincular con GitHub:

Usa el comando:

```
$ cat ~/.ssh/id_ed25519.pub
```

Copia la salida en texto y pégala en los ajustes de GitHub (Settings > SSH and GPG keys).

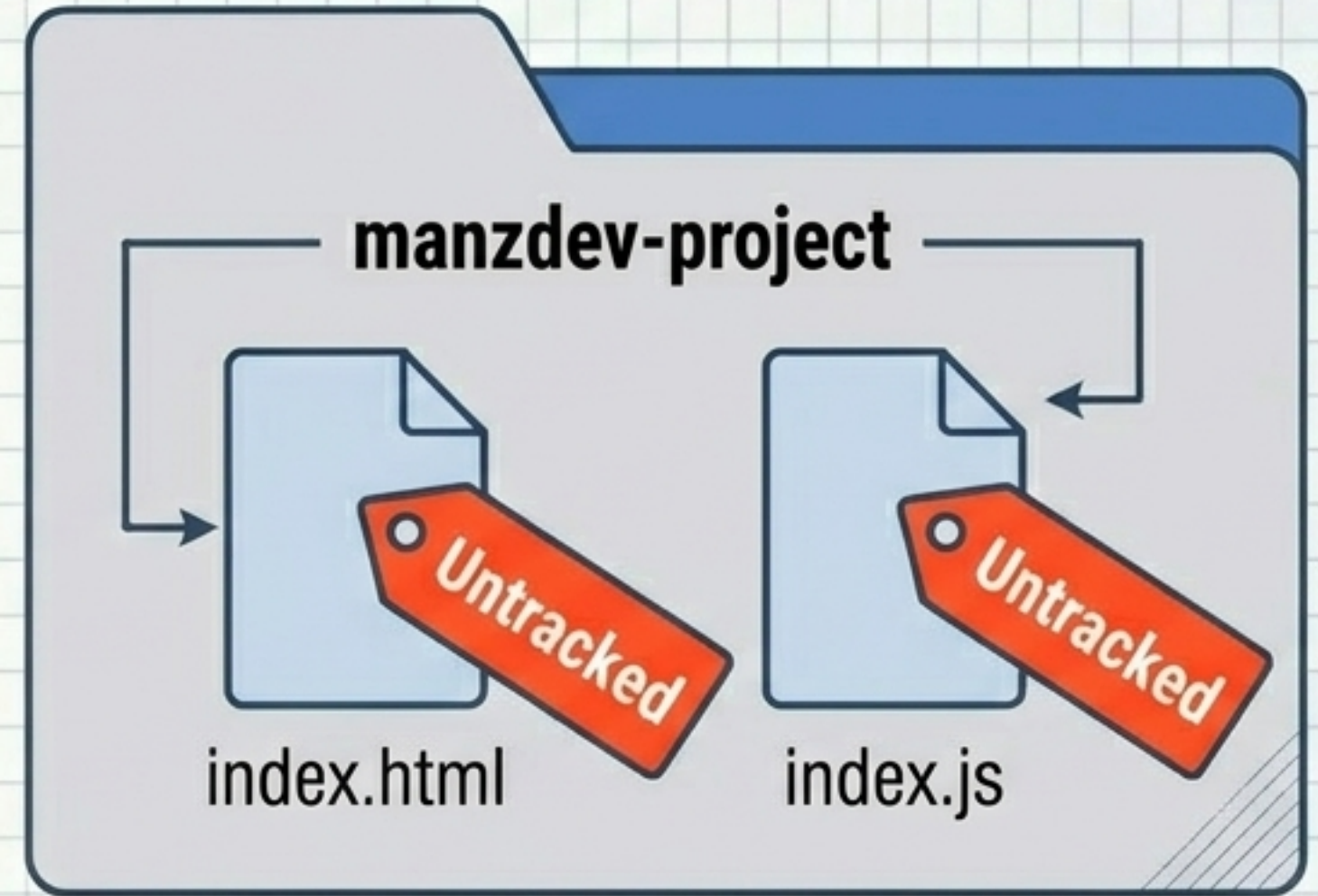
Dos rutas principales para conectar tu código con la nube



Ruta A: Inicializa un proyecto desde cero en tu propia máquina

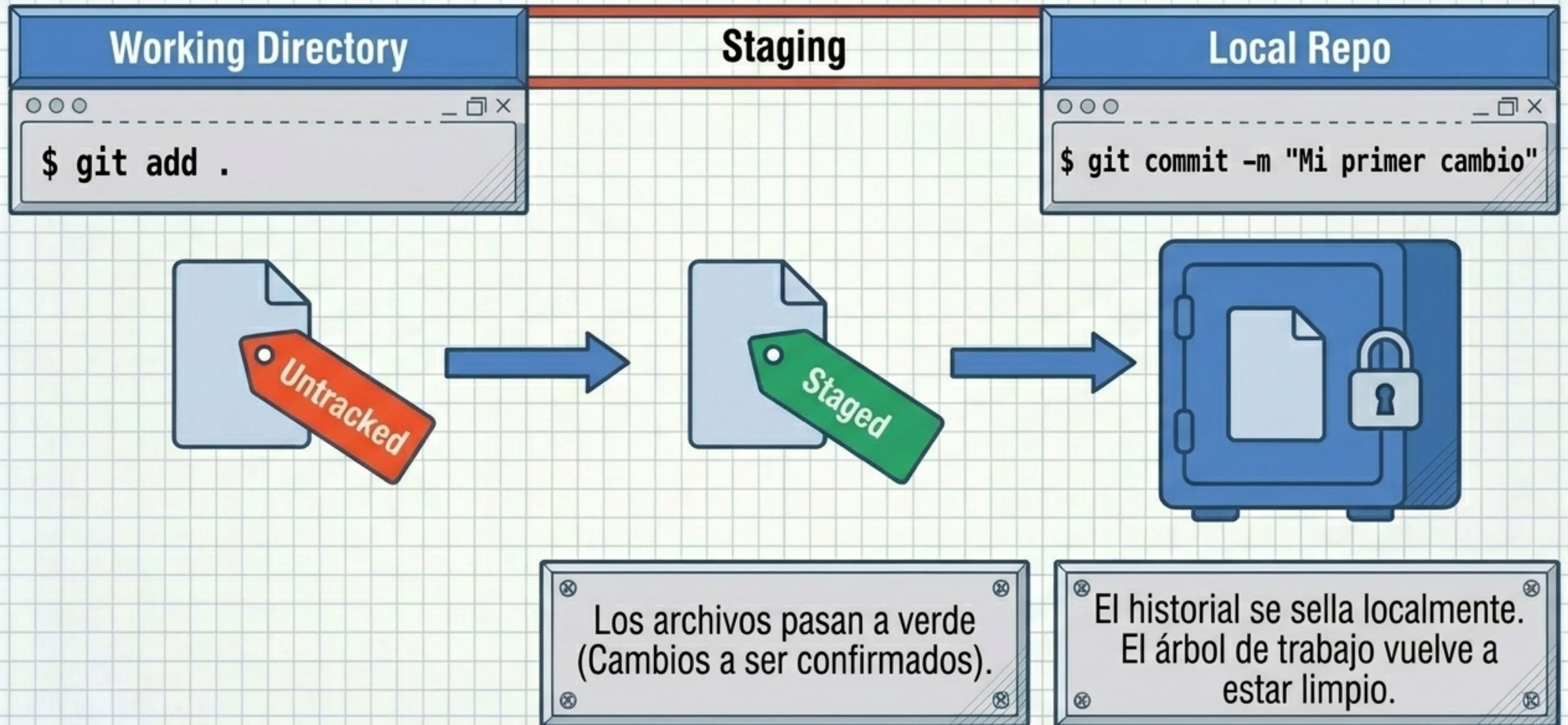
Configurar el Workspace

```
$ mkdir manzdev-project  
$ cd manzdev-project  
$ touch index.html index.js  
$ git init
```



Al ejecutar **git init**, se crea una carpeta oculta **.git/** que comienza a vigilar el directorio. Un **git status** mostrará los archivos en rojo, indicando que aún no tienen seguimiento.

Ruta A: Prepara los archivos y sella los cambios (Add & Commit)



Ruta A: Construye el puente hacia GitHub y sube tu código

1. Vincular

```
$ git remote add origin  
git@github.com:usuario/proyecto.git
```

"origin" es el nombre estándar de nuestro destino en la nube.

2. Desplegar

```
$ git push -u origin main
```

Sube los cambios y establece la ruta por defecto.



Local Laptop



GitHub Cloud

Ruta B: Descarga un repositorio existente desde la nube

1. En la Web:

Crea el repositorio visualmente desde la interfaz de GitHub.

2. En la Terminal:

```
$ git clone git@github.com:usuario/repo.git
```

Insight Crítico: Al clonar, Git descarga el código, realiza el git init automáticamente y deja configurado el puente remoto (origin).
¡No necesitas hacer init ni remote add!



Local Laptop



GitHub Cloud

Ruta B: Modifica, confirma y sincroniza en un ciclo continuo

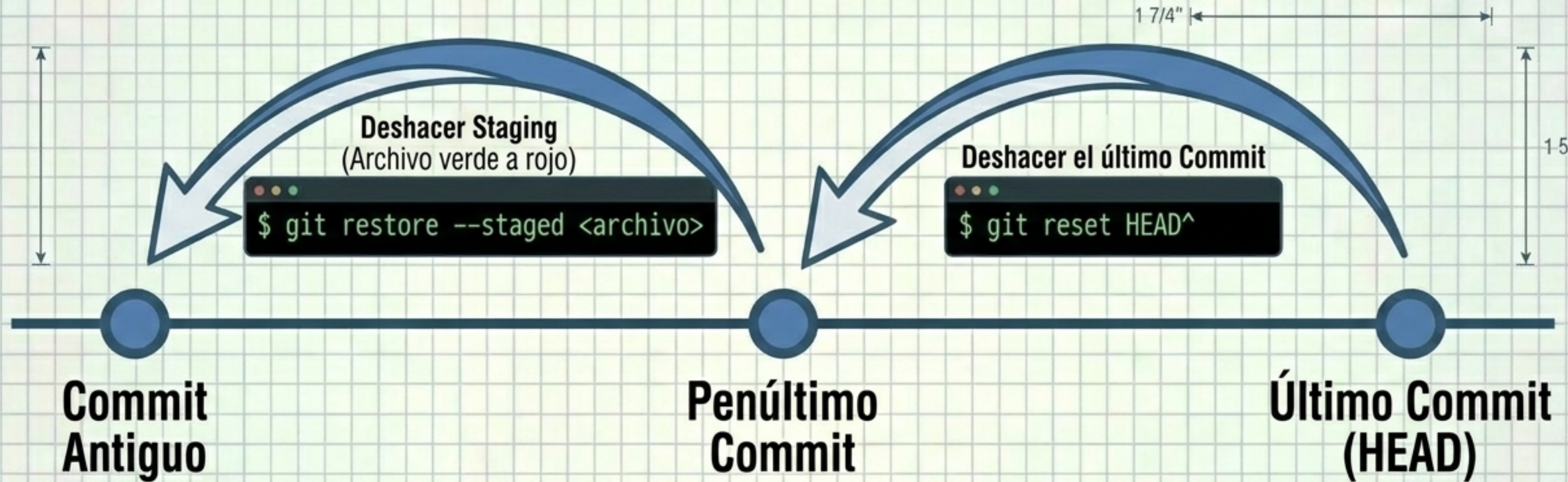


Como ya se configuró el puente al clonar, basta con usar **git push** sin parámetros adicionales.

Matriz Comparativa: Elige tu flujo de trabajo según tu inicio

Dimensión	Ruta A: Local → Remoto	Ruta B: Remoto → Local
Inicio del proyecto	Creando carpetas en tu PC.	Clic en "New Repo" en GitHub.
Secuencia de comandos	mkdir → init → add → commit → remote add → push	clone → add → commit → push
Configuración remota	Se establece manualmente (remote add origin).	Se configura automáticamente al clonar.
Caso de uso ideal	Proyectos locales que ya tienen código.	Proyectos nuevos o contribuciones a código ajeno.

Arqueología y Corrección: Deshaz errores de forma segura



Comando de Diagnóstico: `$ git status`

Es la herramienta más importante para saber exactamente dónde estás y qué archivos están modificados.

Cheat Sheet Visual: Los comandos esenciales por categoría

Configuración

```
$ git config --global user.name  
$ git clone <url>  
$ git remote add origin <url>
```

Trabajo Diario

```
$ git status  
$ git add <archivo> (o add .)  
$ git commit -m "mensaje"  
$ git push" / $ git pull
```

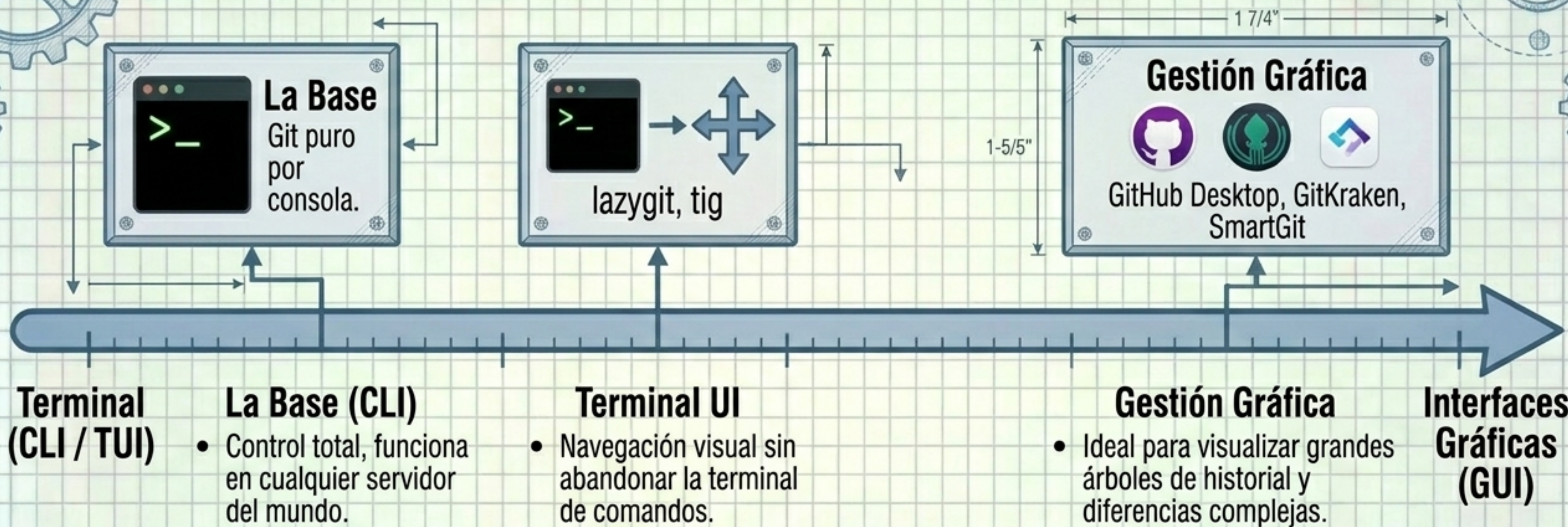
Historial y Auditoría

```
$ git log --oneline  
(Ver historial resumido)  
$ git diff  
(Ver qué líneas han cambiado)
```

Ramas (Branches)

```
$ git branch  
(Listar ramas disponibles)  
$ git switch -c <nombre>  
(Crear y cambiar a una nueva rama)
```


El Ecosistema Git: De la terminal pura a las interfaces visuales



Recomendación: Entiende primero el mapa de rutas en la terminal; luego, elige la herramienta que potencie tu velocidad.